CS 422 Assignment 2: iPerf, TCP Statistics, and Congestion Control

Joseph Getachew, Kushagr Khanna, Isaac Hallman

October 21, 2026

Repository: https://github.com/krash3125/cs422-assignment-2. Code links below point at commit `725adf6`.

Where the measurements ran. Ubuntu 24.04.5 VM (Multipass/QEMU on an Apple M1 Pro Mac, 2 vCPU, 4 GB), host kernel **`6.8.0-147-generic`**. The VM reaches the Internet through macOS packet-level NAT (vmnet) over home Wi-Fi, so every TCP connection runs end-to-end from the VM's kernel to the iperf3 server. The pipeline runs inside Docker (`FROM ubuntu:24.04`) with `--network host --cap-add NET_ADMIN`. `tcp_available_congestion_control = reno cubic bbr` (after `modprobe tcp_bbr`), default algorithm `cubic`, default qdisc `fq_codel`, send buffer limits `tcp_wmem = 4096 16384 4194304` (Ubuntu default). `getsockopt(TCP_INFO)` returns the full 248-byte struct on this kernel.

I did not use Docker Desktop on macOS for the measurements. Its userspace network proxy (vpnkit) terminates the container's TCP connection inside the VM. In a smoke test there, srtt was about 1 ms and cwnd sat at a fixed maximum for every server, which describes the proxy link, not the Internet path.

Exact command: `./run.sh -n 10 -t 30 -i 0.2 -r 3 --seed 422`. That is 10 destinations, 30 s per test, 200 ms sampling, 3 repetitions, and CUBIC, Reno and BBR. Two follow-up runs on 4 destinations, which vary only the send buffer cap, are in `results/wmem32m/` and `results/wmem4m/` (Q2c, Q3a). Raw data for the main run is in `results/20261021-082724/` (started 08:27, finished 09:17 UTC on Oct 21 2026, i.e. 04:27–05:17 ET).

1. iPerf Throughput Application

(a) Python iperf3 client from scratch

The client is `iperf_client.py`. It only uses Python's `socket`, `select`, `struct` and `json` modules; I did not use or call the iperf3 binary. I derived the protocol from the official iperf3 source (`iperf_api.h`, `iperf_client_api.c`, `iperf_api.c`). The steps in `run_test` are:

Step	Implementation
(i) control connection	TCP connect to `host:port`, `TCP_NODELAY`, then send a 37-byte cookie (36 chars from `a–z2–7` + NUL) (`make_cookie`).
(ii) JSON parameter exchange	Wait for state `PARAM_EXCHANGE (9)`, then send the parameter JSON shown below, framed as a 4-byte big-endian length + JSON (`send_json`).
(iii) data connection	Wait for `CREATE_STREAMS (10)`. Open a second socket, set `TCP_CONGESTION` **before** `connect()` (`connect`), send the same cookie, then read `TCP_CONGESTION` back to record the algorithm actually used.
(iv) continuous transmission	After `TEST_START (1)` and `TEST_RUNNING (2)`, `send_loop` sends 128 KiB blocks on a non-blocking socket for the configured duration.

Step	Implementation
(v) termination	Send `TEST_END (4)`. On `EXCHANGE_RESULTS (13)`, send the client results JSON (`cpu_util_*`, `sender_has_retransmits`, `streams[{id:1, bytes, retransmits, …}]`) and read the server's results JSON (`recv_json`). On `DISPLAY_RESULTS (14)`, answer `IPERF_DONE (16)` and close.

Parameters sent in step (ii) (`time` is the test duration in seconds, `len` the block size):

```
{"tcp": true, "omit": 0, "time": 30, "num": 0, "blockcount": 0, "parallel": 1,
 "len": 131072, "pacing_timer": 1000, "client_version": "3.17"}
```

Robustness. Every failure becomes a typed `status` on the returned `TestResult` instead of an exception, so one bad server never stops the pipeline.

Situation	Detection	Status
Non-responsive / overloaded server	no state byte within 15 s on the control socket (`recv_exact`)	`CTRL_TIMEOUT`
Server busy with another client	state `ACCESS_DENIED (-1)`, or `SERVER_ERROR` with `i_errno 121`	`ACCESS_DENIED`
Server rejects our parameters	state `SERVER_ERROR (-2)` followed by `i_errno`/`errno` (e.g. 37 = duration above server max, 27 = rate above limit) (`read_state`)	`PARAM_REJECTED` / `SERVER_ERROR`
Connection timeouts	5 s connect timeout on both sockets	`CONNECT_TIMEOUT`
Refused / unreachable / DNS	`ECONNREFUSED`, other `OSError`, `gaierror`	`CONNECT_REFUSED`, `CONNECT_ERROR`, `DNS_FAIL`
Premature termination	EOF/reset on the control socket or a send error on the data socket, checked every loop iteration via `select()`; `SERVER_TERMINATE (11)`	`PREMATURE_CLOSE`, `SERVER_TERMINATE`
Algorithm not available	`setsockopt(TCP_CONGESTION)` fails	`CC_UNAVAILABLE`

Partial runs keep their samples (`complete=false`). The runner retries transient failures with a growing backoff (`run_with_retries`), and for multi-port servers it tries a different port in the range. I checked the error paths by hand: a non-routable IP gives `CONNECT_TIMEOUT`, a filtered port gives `CONNECT_TIMEOUT`, and an invalid name gives `DNS_FAIL`.

Validation against the server. iperf3 servers report how many bytes they received in their results JSON. In all 90 tests the server's `streams[0].bytes` matched our final `tcpi_bytes_acked` to within 145 KB out of about 650 MB (at most 0.07%, median 0.01%). The small gap goes both ways, so it comes from where each side stops counting at the end of the test, not from lost data. This confirms the protocol exchange end to end and shows that acknowledged bytes measure what the server actually received. Every test reported `complete = true` and lasted 30.00 s.

(b) Destination selection

The input file `servers.csv` is a snapshot of the iperf3serverlist.net JSON export (183 servers). It can be refreshed with `--refresh-servers`. `select_destinations` shuffles the list with a seeded RNG (`--seed 422`) and runs a 3 s probe test against each candidate. If the probe fails because the port is busy or refused, it tries one more port of the same host. Any other failure moves on to a replacement server. It stops after n successes or `--max-attempts` (40) probes, and reports any shortfall in `destinations.json`. Every attempt is logged in `attempts.csv`.

#	Destination	IP	Port	Site	min RTT (ms)
1	speedtest.wdc2.us.leaseweb.net	23.105.9.80	5209	Washington, US	26.2
2	speedtest.dal13.us.leaseweb.net	209.58.159.68	5207	Dallas, US	27.0
3	66.35.22.79	66.35.22.79	30003	Ashburn, US	27.2
4	84.17.41.11	84.17.41.11	5201	Seattle, US	52.7
5	iperf3.moji.fr	45.147.210.189	5232	Paris, FR	96.9
6	iperf-ams-nl.eranium.net	217.18.95.142	5202	Amsterdam, NL	102.2
7	iperf.worldstream.nl	185.182.195.76	5203	Naaldwijk, NL	104.7
8	89.187.165.1	89.187.165.1	5201	Zürich, CH	116.7
9	speedtest.syd12.au.leaseweb.net	23.106.244.70	5202	Sydney, AU	180.7
10	89.187.162.1	89.187.162.1	5201	Singapore, SG	207.7

Outcome: 10 of 10 selected, shortfall 0, 12 probes. (`destinations.json`, `tables/q1_destinations.md`, `tables/q1_failures.md`.) Two candidates failed during selection:

- `122.228.230.122:5205` accepted the TCP connection but never sent a state byte. After 15 s it was marked `CTRL_TIMEOUT` (non-responsive server) and replaced with the next server in the shuffled list.
- `iperf3.moji.fr:5204` answered `ACCESS_DENIED` (busy). The second random port, 5232, worked.

During the 90 measurement tests, 6 attempts failed transiently: 3 × `ACCESS_DENIED` and 3 × `PREMATURE_CLOSE`, all on the two Asia-Pacific servers. The worst case was Sydney BBR in rep 2, which needed four attempts. All of them succeeded on retry against the same host (on another port when the host has a range), so **all 90 tests are valid and every algorithm was measured on the same 10 destinations.**

(c) Acknowledged throughput (goodput)

Every 200 ms, `send_loop` reads `TCP_INFO` from the **data** socket with `getsockopt()` and `make_sample` computes

$$\text{goodput}(t_k) = \frac{\texttt{tcpi_bytes_acked}(t_k) - \texttt{tcpi_bytes_acked}(t_{k-1})}{t_k - t_{k-1}} \cdot 8 \text{ [bit/s]}$$

where t_k comes from `time.monotonic()` at the moment of the actual read, not the nominal schedule. The loop waits in `select()` with a timeout capped at the next sample deadline, so sampling stays on time even when the socket buffer is full. The median measured interval was 0.200 s (1st–99th percentile 0.138–0.263 s), and every 30 s test produced 147–150 samples. Because the counter is *acknowledged* bytes, data still queued in the socket buffer never counts. Plots: `q1_goodput_plots`; table: `q1_summary`.

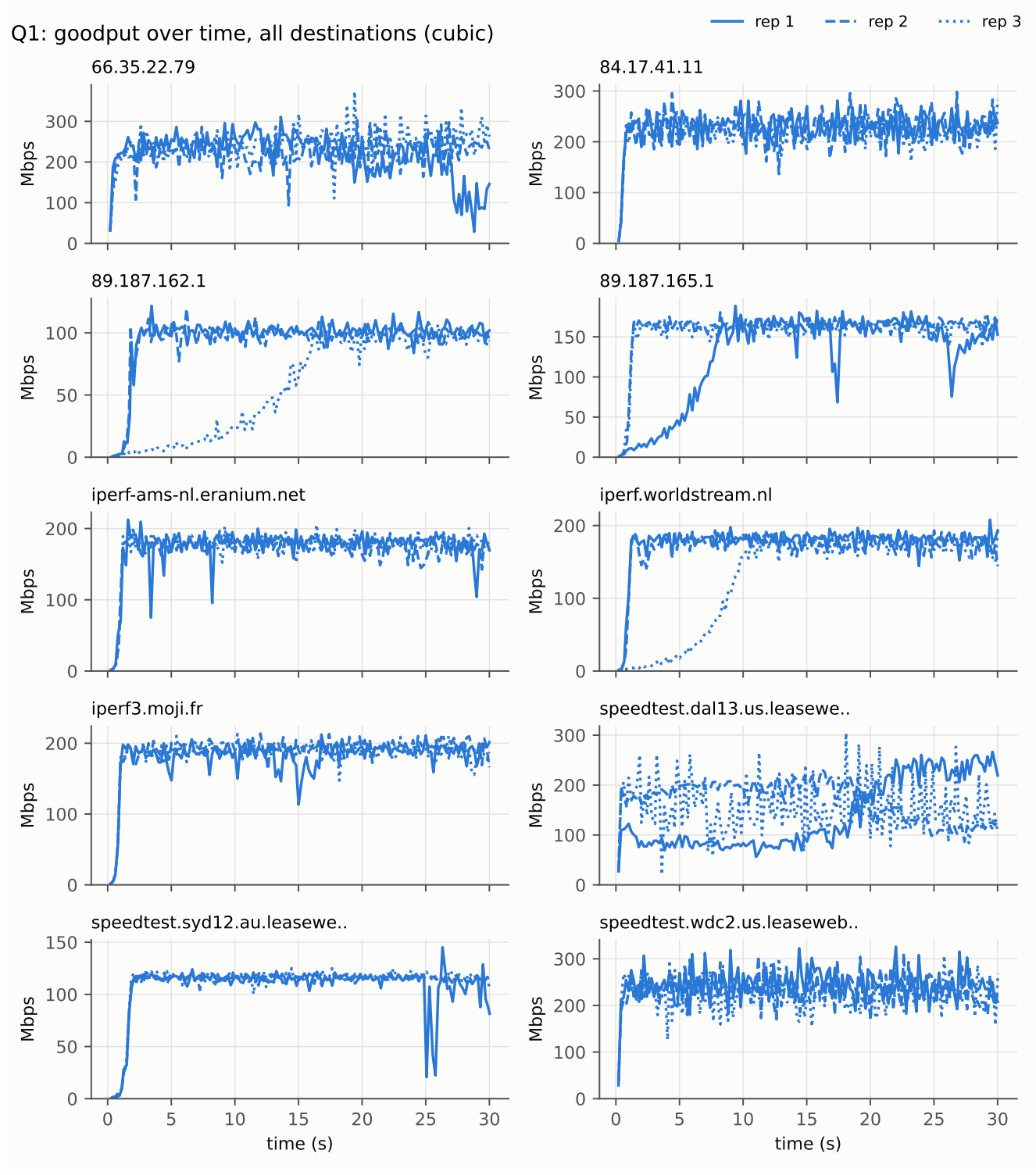


Figure 1: Q1 goodput every 200 ms, CUBIC. One panel per destination, one line style per repetition.

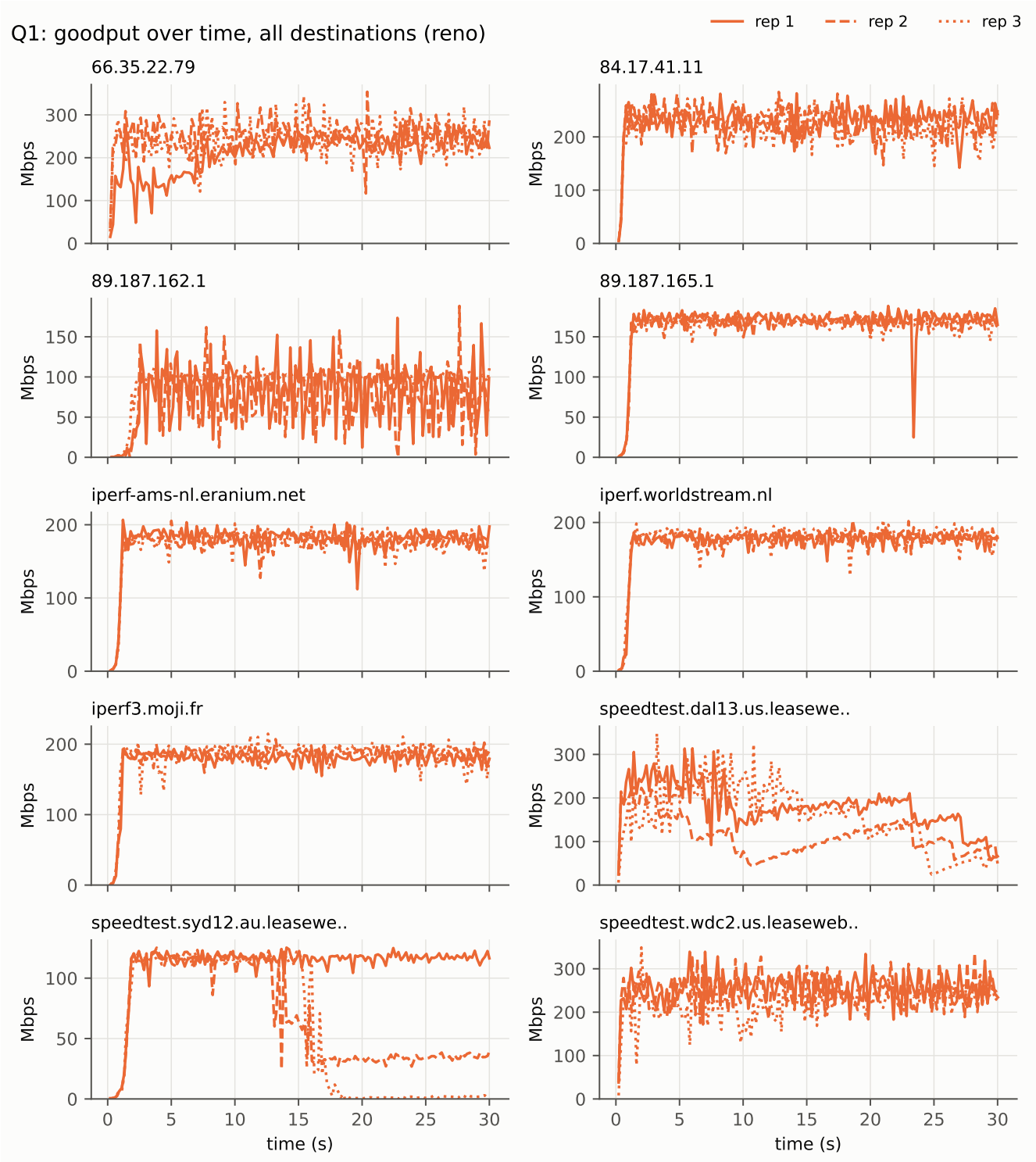


Figure 2: Q1 goodput, Reno.

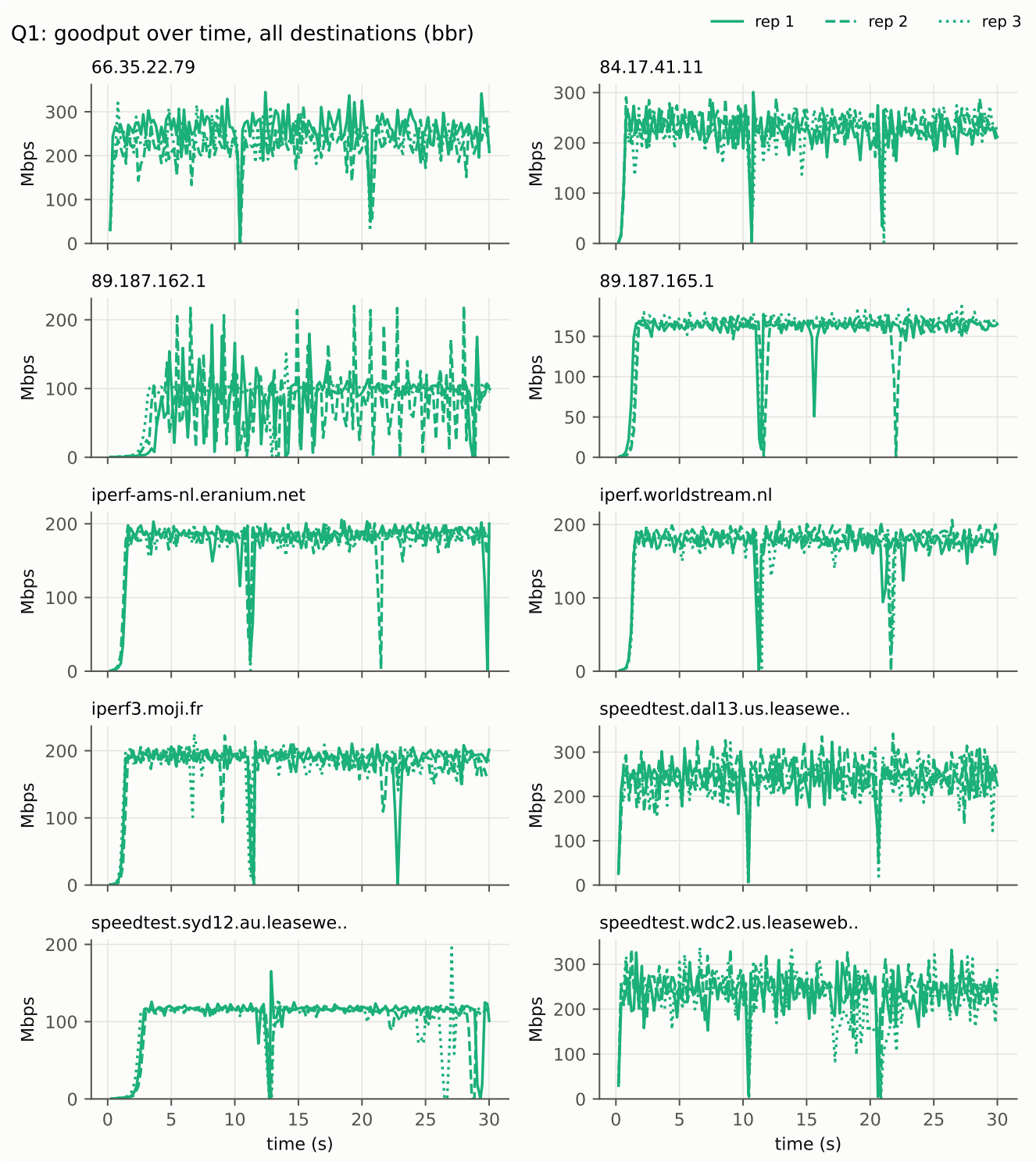


Figure 3: Q1 goodput, BBR.

Goodput summary over all 200 ms samples of the 3 repetitions (450 samples per row, Mbps; `tables/q1_goodput_summary.md`). Destinations are sorted by min RTT and named by city (hostnames are in the selection table above):

Destination (min RTT)	CC	min	median	mean	p95
Washington (26 ms)	cubic	28.7	233.5	233.7	289.3

Destination (min RTT)	CC	min	median	mean	p95
	reno	8.1	242.4	241.8	302.2
	bbr	0.4	241.2	234.5	306.8
Dallas (27 ms)	cubic	20.3	152.8	154.4	246.8
	reno	5.7	149.2	149.5	260.4
	bbr	1.5	244.5	240.1	304.8
Ashburn (27 ms)	cubic	28.7	234.3	228.4	291.9
	reno	15.4	241.1	235.3	296.5
	bbr	1.2	245.8	239.7	303.1
Seattle (53 ms)	cubic	4.4	229.1	225.0	266.8
	reno	3.8	227.9	224.6	267.1
	bbr	0.5	228.8	223.6	269.9
Paris (97 ms)	cubic	1.1	190.1	184.7	205.5
	reno	0.5	185.9	180.4	198.9
	bbr	0.5	189.9	178.6	205.0
Amsterdam (102 ms)	cubic	1.0	178.7	172.7	193.6
	reno	0.6	181.3	175.1	194.5
	bbr	0.5	184.5	174.0	197.8
Naaldwijk (105 ms)	cubic	0.6	176.6	158.8	190.4
	reno	1.0	180.6	174.8	192.2
	bbr	0.3	179.9	170.2	195.4
Zürich (117 ms)	cubic	0.5	162.7	148.1	173.8
	reno	0.5	169.5	164.1	179.7
	bbr	0.3	165.5	155.8	176.3
Sydney (181 ms)	cubic	0.4	115.8	110.0	121.4
	reno	0.0	113.3	78.9	121.6
	bbr	0.0	115.0	102.4	120.6
Singapore (208 ms)	cubic	0.0	97.9	82.7	107.5
	reno	0.0	97.4	84.3	124.1
	bbr	0.0	97.1	83.1	146.4

Goodput ranges from about 80 Mbps (Singapore) to about 240 Mbps (US East). The minimum column is the first sample of slow start, or a stall. The 0.0 values occur only on the 180–210 ms paths. Most are in the first second, when a 200 ms interval is shorter than one round trip and no ACK can arrive yet. The rest are stalls during BBR ProbeRTT or loss recovery. Most series settle within 1–2 s and then stay flat. The exceptions are explained in Q2(c) and Q3(a): HyStart exiting slow start early (slow CUBIC ramps at Singapore, Zürich and Worldstream), Reno's sawtooth at Dallas, and Reno's collapse after a loss burst at Sydney.

2. TCP Stats Tracing During Transfer

All Q2 measurements use **CUBIC**, the kernel default (`tcp_congestion_control = cubic`). Each test reads the algorithm back from the data socket, and every Q2 sample has `cc_used = cubic`.

(a) Periodic TCP statistics

Decoding format (kernel 6.8, x86-64/arm64 little-endian). `tcp_info.py` lists the fields of `struct tcp_info` from `include/uapi/linux/tcp.h` in kernel order with their C types, and `decode` unpacks them with `struct.unpack_from("=<type>")` until the returned length runs out. Older kernels return a shorter struct, so the newest fields are simply absent and the decoder never misreads them. All fields are naturally aligned, so unpacked offsets match the C layout. On this kernel the length is 248 bytes. The fields we use:

Offset	Type	Field	Logged as
1	u8	`tcpi_ca_state`	`ca_state` (0 Open, 1 Disorder, 2 CWR, 3 Recovery, 4 Loss)
7	u8	flags (bit 0 = `delivery_rate_app_limited`)	`app_limited`
16	u32	`tcpi_snd_mss`	`mss_bytes`
32	u32	`tcpi_lost`	`lost_pkts`
68 / 72	u32	`tcpi_rtt` / `tcpi_rttvar` (µs)	`srtt_ms`, `rttvar_ms`
80	u32	`tcpi_snd_cwnd` (segments)	`cwnd_pkts`, `cwnd_bytes = cwnd × mss`
100	u32	`tcpi_total_retrans`	`retrans_pkts_delta` (per interval)
104	u64	`tcpi_pacing_rate` (B/s)	`pacing_rate_bps`
120	u64	`tcpi_bytes_acked`	goodput, `bytes_acked`
148	u32	`tcpi_min_rtt` (µs)	`min_rtt_ms`
160	u64	`tcpi_delivery_rate` (B/s)	`delivery_rate_bps`
168–184	u64	`busy_time`, `rwnd_limited`, `sndbuf_limited` (µs)	`rwnd_limited_frac`, `sndbuf_limited_frac`
200 / 208	u64	`tcpi_bytes_sent` / `tcpi_bytes_retrans`	`retrans_bytes_delta`, `retrans_ratio`
240	u16	`tcpi_total_rto`	`rto_events_delta` (timeouts per interval)

Interval measurements. Each sample stores the decoded snapshot. Every rate or count is the difference between consecutive snapshots divided by their measured time gap (`make_sample`). cwnd, srtt, rttvar, min_rtt, `lost`, `ca_state`, pacing and delivery rate are instantaneous values.

Loss proxy. The main loss signal is `retrans_pkts_delta`: segments retransmitted during the interval (fast retransmits + RTO retransmits, from `tcpi_total_retrans`). Alongside it I log retransmitted bytes / bytes sent per interval, timeouts per interval (`tcpi_total_rto`), the kernel's current lost-segment estimate (`tcpi_lost`), and whether the connection was in Recovery/Loss state. Retransmissions overcount losses slightly, since spurious retransmits are also counted. I accepted that because the receiver side, which would see the true losses, is not ours.

Storage. `samples.csv` has one row per sample with `run_id`, `test_id`, `rep`, `seq`, destination (host/IP/port), `cc_requested`, `cc_used`, and the metrics. Every column name carries its unit (`_bps`, `_ms`, `_pkts`, `_bytes`, `_s`), and `metadata.json` repeats the units. `tests.jsonl` holds per-test summaries plus the server's JSON. The writer is `run_all`.

(b) Visualization

The representative destination is chosen automatically (`pick_rep_dest`): among destinations where every algorithm × repetition succeeded, it is the one with the median CUBIC goodput. For time series we plot the run with the median mean goodput (`representative_run`), and the scatters use all three CUBIC repetitions. Code: `q2_timeseries`, `draw_timeseries`, `q2_scatter`.

The representative destination picked automatically is **iperf-ams-nl.eranium.net** (Amsterdam, min RTT 102 ms), whose CUBIC goodput is the median of the 10 destinations. The time-series plot shows its median CUBIC run (rep 3).

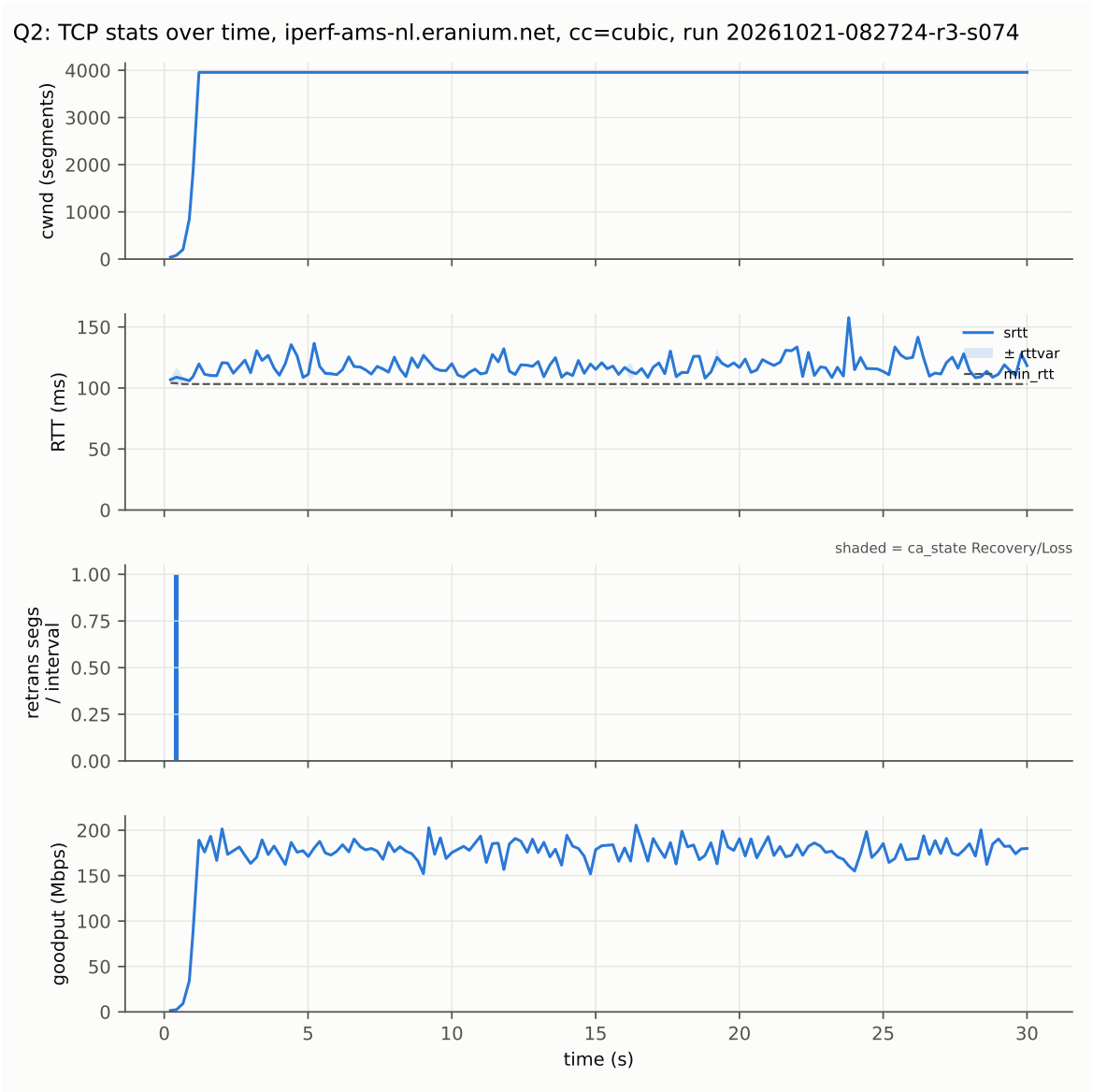


Figure 4: Q2 time series, CUBIC to Amsterdam: cwnd, srtt (band = ±rttvar, dashed = min_rtt), retransmitted segments per 200 ms (shaded = ca_state Recovery/Loss), and goodput.

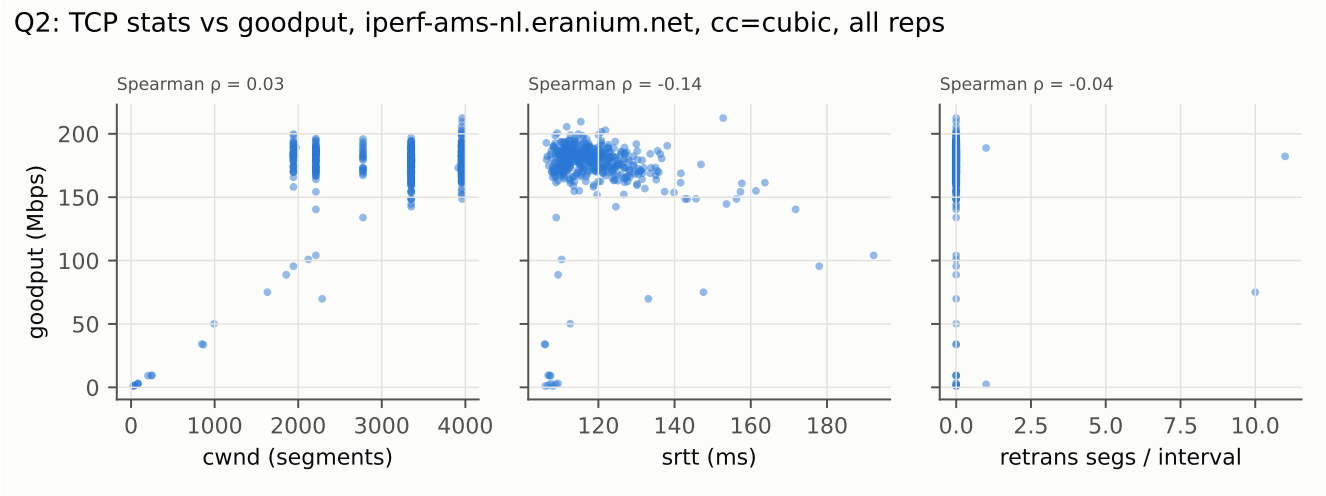


Figure 5: Q2 scatter, all 3 CUBIC runs to Amsterdam (one dot per 200 ms sample). Spearman ρ in each panel.

(c) Observations

What each metric means.

- **cwnd** (`tcpi_snd_cwnd`, segments of 1448 B) is how many unacknowledged segments the congestion controller allows in flight.
- **srtt** is the kernel's smoothed RTT and **min_rtt** is the lowest RTT seen, which approximates the propagation delay. srtt − min_rtt is the queueing delay this flow sees.
- **Retransmitted segments per interval** is the loss proxy. A non-zero value, together with `ca_state` = Recovery, means the sender detected a loss and is repairing it.
- **Goodput** is the rate at which the receiver acknowledges new data.

A window-based sender delivers about one window per round trip: goodput $\approx$ in-flight/srtt, where in-flight = min(cwnd, receiver window, what the send buffer holds).

What happens in congestion avoidance (general case). If cwnd is what limits in-flight data and it is below the BDP, each increase in cwnd raises goodput. Once cwnd passes the BDP the link is full, so extra window only lengthens the bottleneck queue: srtt rises above min_rtt and goodput stays flat. When the buffer overflows, a loss follows: a retransmit and Recovery state. CUBIC then cuts cwnd to 0.7×, the queue drains, and srtt falls. The cwnd → RTT → loss → cwnd loop is what Q3 compares across algorithms.

What this trace shows.

1. *Slow start.* cwnd grows from 10 to about 3950 segments in about 1.3 s, and goodput rises with it. The only points where goodput tracks cwnd closely are these early samples (lower left of the cwnd scatter, where goodput ≈ cwnd·MSS/RTT). Slow start ends with a single retransmitted segment at 0.4 s. CUBIC sets ssthresh = cwnd and enters congestion avoidance.
2. *Steady state is not cwnd-limited.* After slow start, cwnd stays **perfectly flat** for 29 s, and goodput holds steady at about 175 Mbps. Over the three runs the median in-flight data is **2.53 MB = 1750 segments**, well below cwnd (3350–3950 segments). The cause is the sender's socket buffer, not the network:
   - `tcp_wmem` max is 4 MiB on this host, and the unacknowledged data must stay in that buffer.
   - `select()` only reports the socket writable when about a third of the buffer is free.
   - So in-flight data is capped at roughly 2/3 × 4 MiB ≈ 2.6 MB.

   The kernel's own counters agree:
   - `notsent_bytes` is only about 86 KB (the queue is nearly empty).
   - 72% of samples carry the `delivery_rate_app_limited` flag.
   - `rwnd_limited` is about 0.4%. The receiver offered a 15 MB window, so it is not the limit.

   Linux grows cwnd only while the flow is cwnd-limited (`tcp_is_cwnd_limited`), which is why the line is flat. The numbers fit: 2.53 MB × 8 / 0.1166 s = 174 Mbps, compared with 173 Mbps measured.

3. *Loss barely matters here.* In rep 1 (not plotted) CUBIC cut cwnd twice, 3962 → 2774 → 1943, exactly ×0.7 each time. Its goodput (173.9 Mbps) was still the same as the runs without cuts (170.6 and 173.5 Mbps), because even 1943 segments (2.8 MB) is above the 2.6 MB send-buffer cap. This is why the Spearman correlations are near zero: cwnd vs goodput $\rho = 0.03$, retransmits vs goodput $\rho = -0.04$. The scatter shows vertical stripes, one per cwnd level, all at the same goodput.
4. *RTT and queueing.* srtt has a median of 116.6 ms against a min_rtt of 102.7 ms, so there is about 14 ms of queue (inflation 1.14). srtt jumps around by a standard deviation of about 9 ms between samples, with spikes to 155 ms. With in-flight data fixed, goodput 1/srtt, which explains the weak negative correlation ($\rho = -0.14$) and the sample-to-sample goodput noise of ±10 Mbps.

Anomalies.

- *Send-buffer limit.* Explained in point 2 above. I kept the distribution's default `tcp_wmem` so the main results describe a stock Ubuntu host. To confirm the cause, I reran 4 destinations with the cap raised to 32 MiB (below).
- *rttvar is tiny.* `tcpi_rttvar` is only about 0.2–0.5 ms even though srtt itself moves by about 10 ms between samples. Linux updates rttvar on every ACK (thousands per RTT here), so it measures the ACK-to-ACK difference, not the slower changes from the Wi-Fi link. That makes it a poor guide to delay noise on this path.
- *Early slow-start exit (HyStart).* On several long paths CUBIC left slow start with no loss at all. For example, Singapore rep 3 exited at ssthresh = 59 segments: HyStart's delay check fired on a Wi-Fi RTT bump. It then took 20 s of CUBIC growth to reach the cap, so that run averaged 57.5 Mbps against 94–96 Mbps for the other two reps. These are the slow ramps in the Q1 CUBIC panels.
- *Pacing rate is not applied.* `tcpi_pacing_rate` (median 391 Mbps) is the rate the kernel computes for CUBIC, but it is not enforced: the qdisc is `fq_codel`, not `fq`, and CUBIC does not ask for internal pacing. `delivery_rate` (median 205 Mbps) is a per-ACK estimate that runs above the 200 ms goodput because ACKs arrive in bursts.

Follow-up: the same path without the send-buffer cap. To check the explanation in point 2, I raised the cap (`sysctl net.ipv4.tcp_wmem="4096 16384 33554432"`) and reran 4 destinations, Amsterdam among them, with the same settings (`results/wmem32m/20261021-135240/`; `metadata.json` records the `tcp_wmem` value). Now cwnd is the limit, and the congestion-avoidance loop described above shows up clearly.

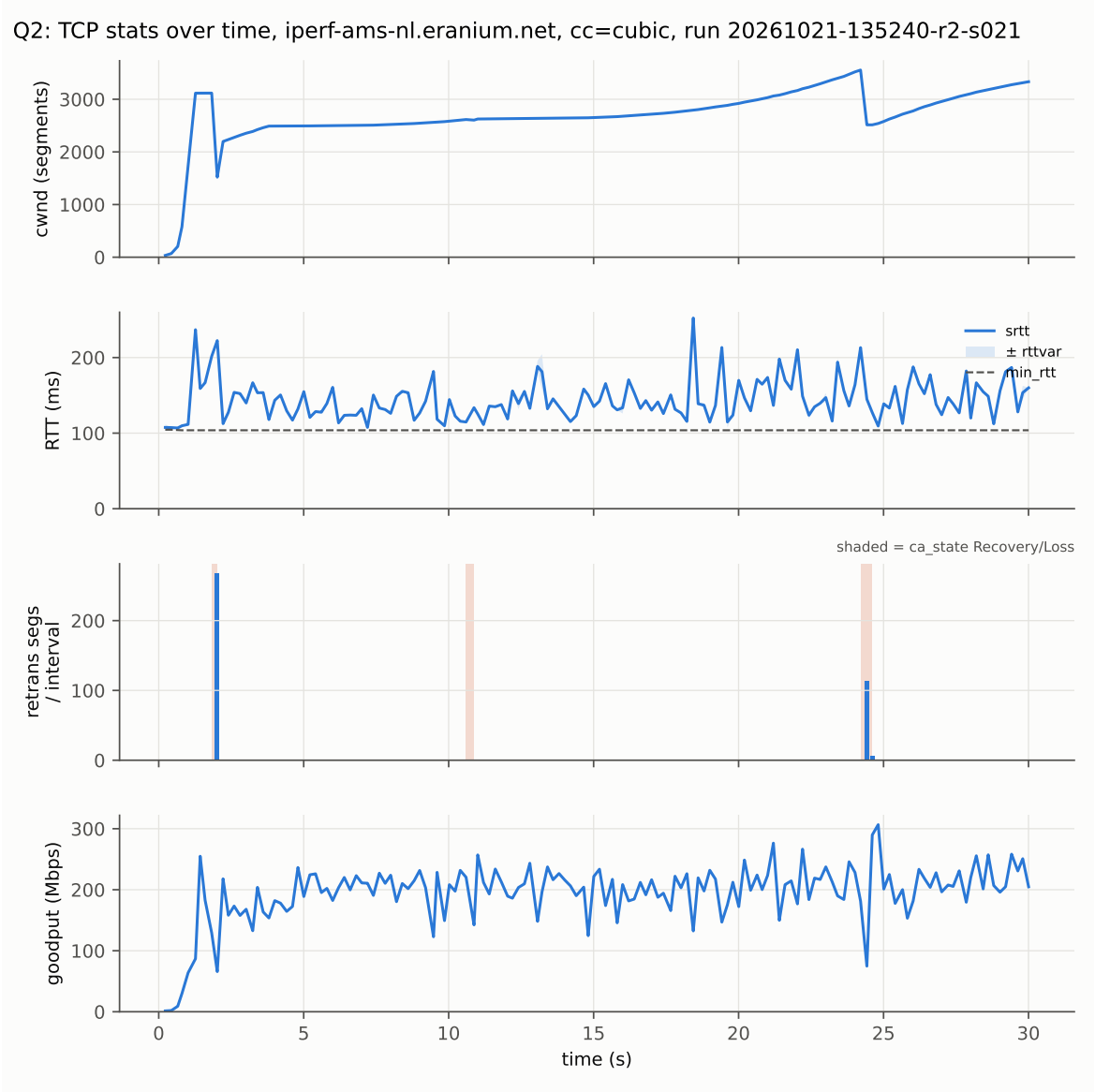


Figure 6: Q2 time series, CUBIC to Amsterdam with `tcp_wmem` max = 32 MiB (median run, rep 2). The send buffer stays full (notsent ≈ 14 MB) and cwnd is the binding limit.

- *Slow-start overshoot.* cwnd reaches about 3100 segments and the bottleneck buffer overflows at 2.0 s. 268 segments are retransmitted, srtt doubles to 222 ms, and CUBIC sets ssthresh = 2182 (×0.7).
- *CUBIC's curve in congestion avoidance.* After the cut, cwnd rises quickly to about 2500, flattens near the old maximum (2500–2640 segments for 4–14 s, the concave part), and then accelerates (the convex probing part) to 3555 at 24.2 s.
- *The cwnd, RTT, loss loop.* As cwnd grows, the queue grows: srtt peaks at 213 ms just before the next loss (113 segments retransmitted at 24.4 s). cwnd then drops to 2514 (about ×0.7), srtt falls back to about 127 ms, and the concave climb starts again.
- *Correlations.* Over the three runs (after the first 3 s), the Spearman correlation is **0.48 between cwnd and srtt** and **0.54 between cwnd and goodput**, against 0.03 for cwnd vs goodput in the capped run.
- *Overall.* Mean goodput rises to 195 Mbps from 173, but srtt rises too (median 139 ms, inflation 1.34) and 0.4% of bytes are retransmitted, against almost none with the cap.

3. Comparing CUBIC, Reno and BBR, and a New Design

(a) Comparison and operating regimes

Method. The same 10 destinations, 30 s per test, and 200 ms sampling for every algorithm. Each test sets the algorithm on the data socket before `connect()` and reads it back. `cc_used` equals `cc_requested` in all successful tests. There are **3 repetitions**. The order is: for each repetition, for each destination, the three algorithms back to back, with the algorithm order rotated each repetition (rep 1: cubic → reno → bbr; rep 2: reno → bbr → cubic; rep 3: bbr → cubic → reno). Each algorithm therefore holds each position once per destination, and the three algorithms on a destination are measured within ~95 s of each other (`run_all`). Plots and tables: `q3_timeseries_grid`, `q3_scatter_grid`, `per_test_metrics`, `q3_dest_summary`, `q3_variability`, `q3_tradeoff`.

Definitions: **RTT inflation** = median srtt / min_rtt, the queueing delay we add. **BDP** = max delivery rate × min_rtt, the data the path holds with an empty queue. **cwnd/BDP** above 1 means the extra window is sitting in a queue.

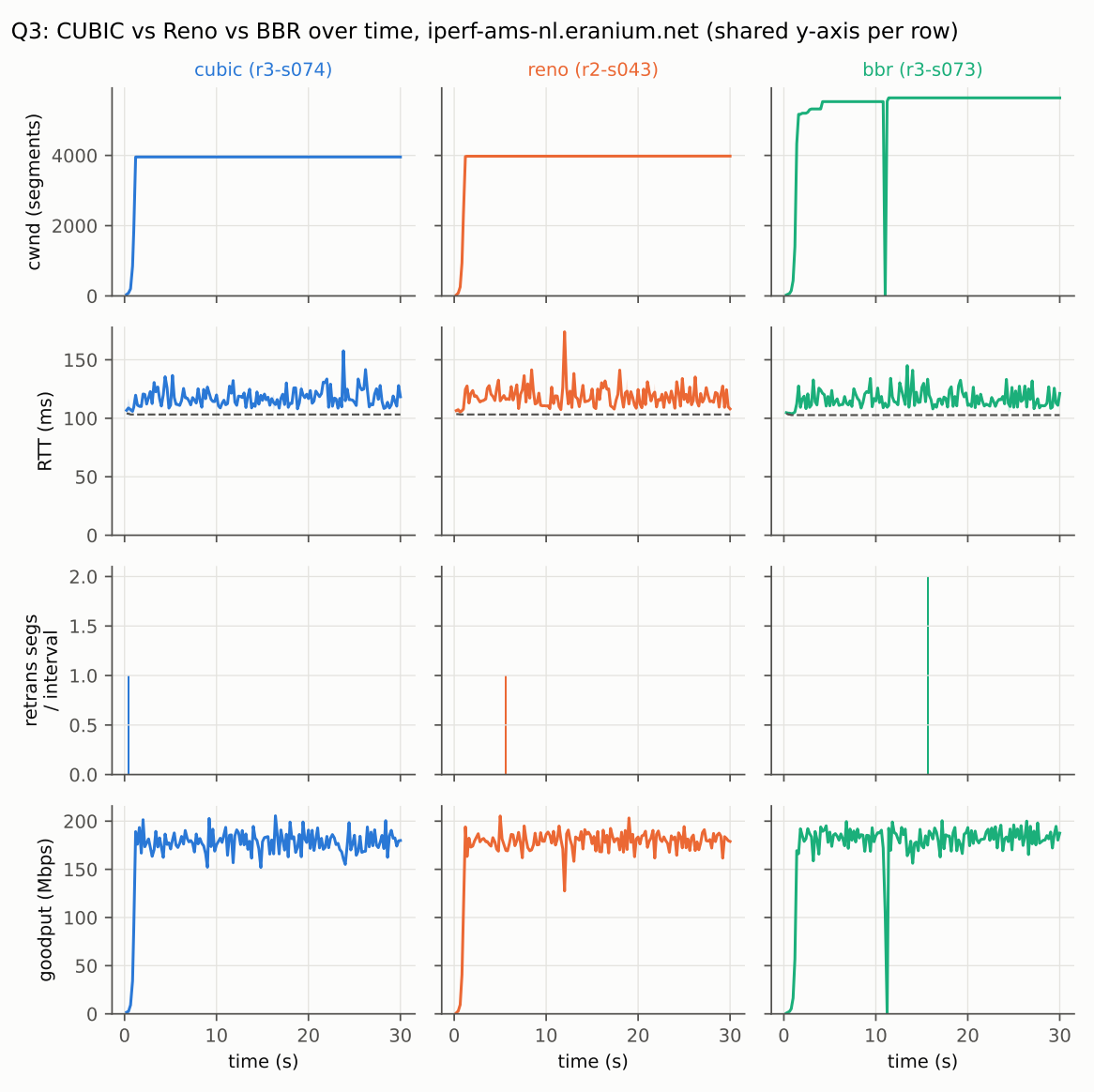


Figure 7: Q3 time series for the representative destination (Amsterdam), median run of each algorithm, shared y-axis per row.

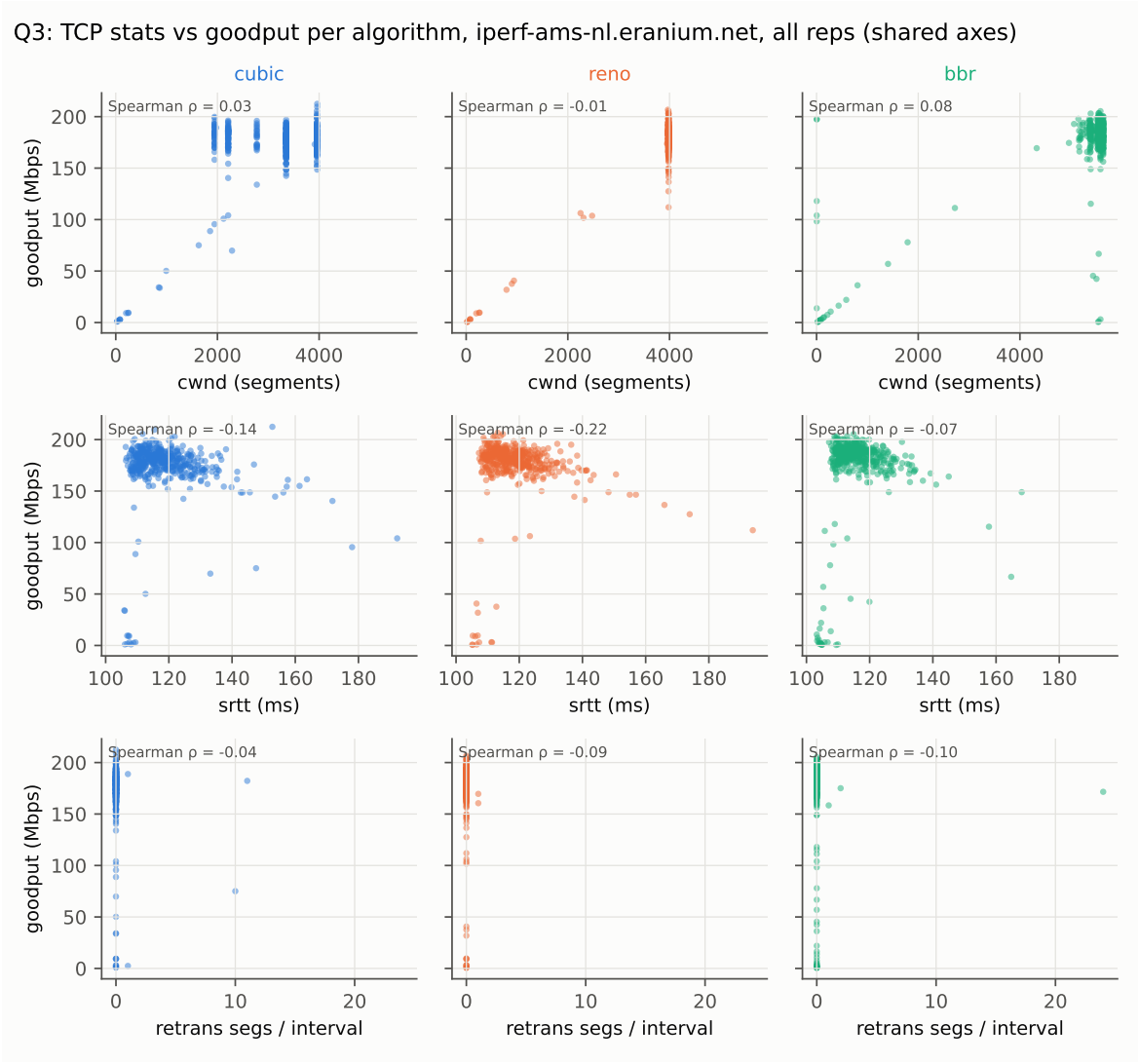


Figure 8: Q3 scatter of cwnd, srtt and retransmits vs goodput for each algorithm (Amsterdam, all reps, shared axes).

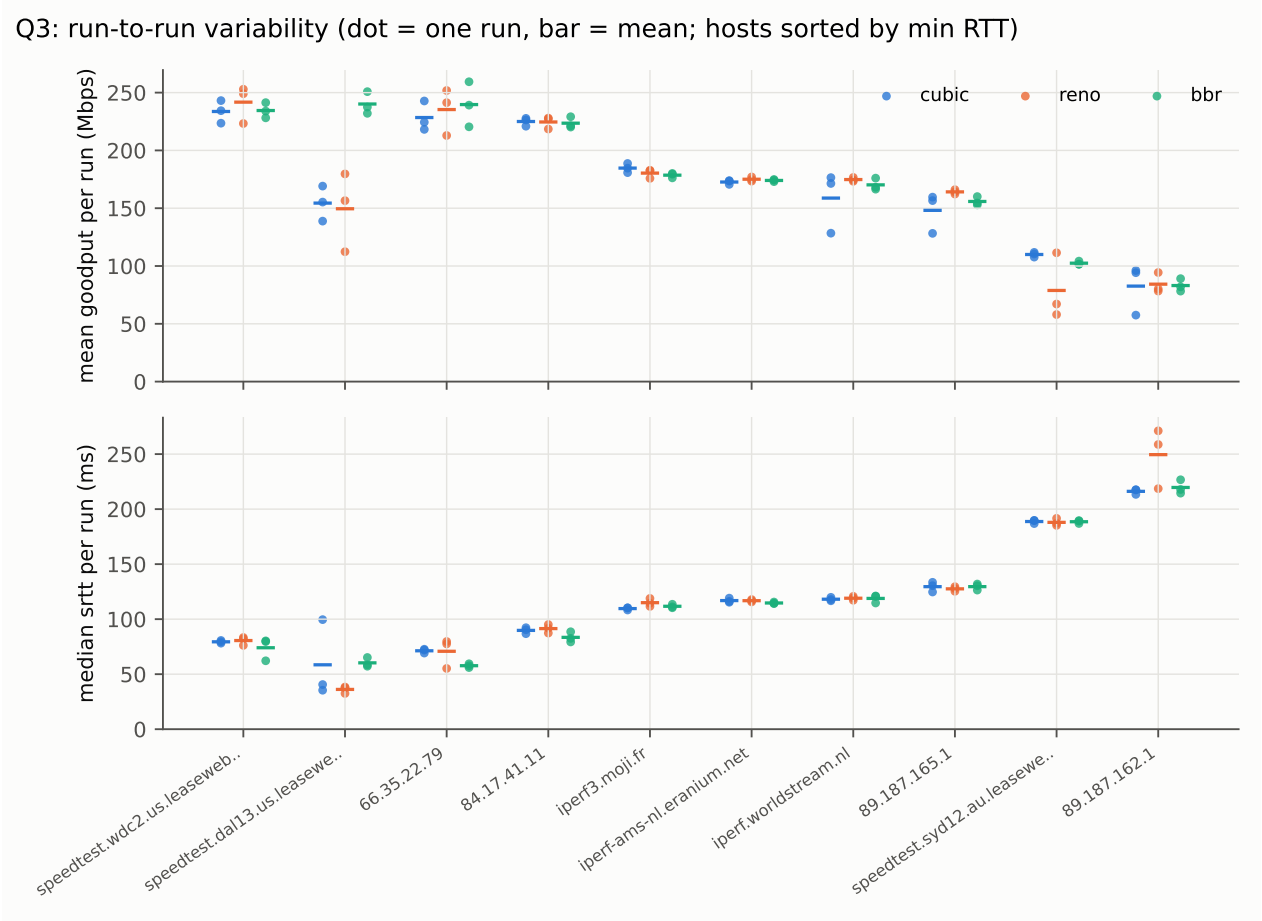


Figure 9: Q3 run-to-run variability: one dot per run, bar = mean of 3 reps, destinations sorted by min RTT.

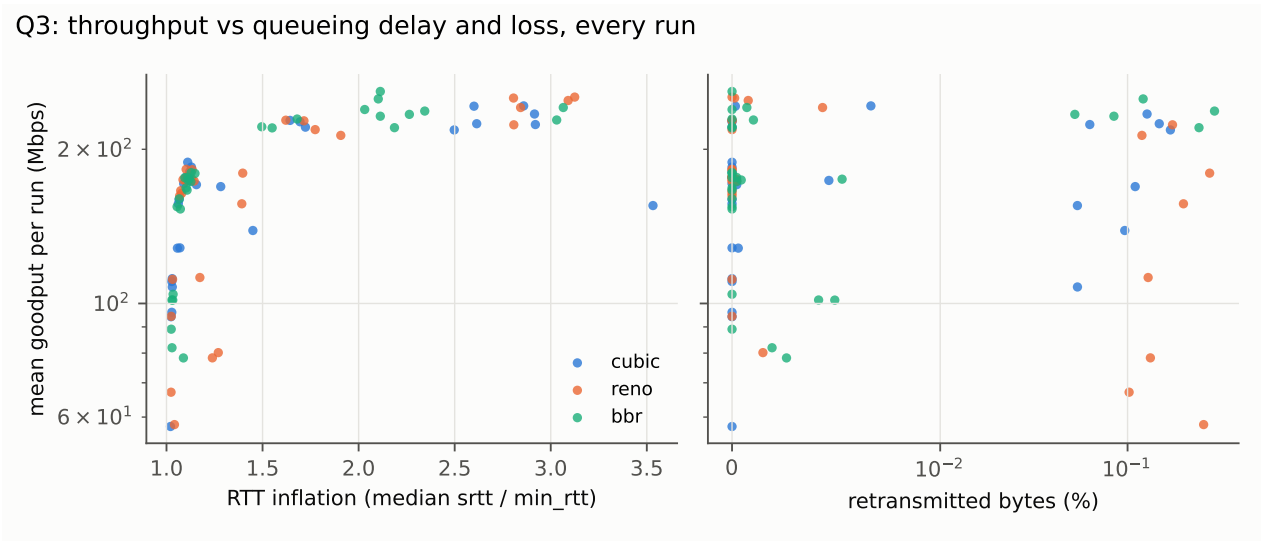


Figure 10: Q3 trade-offs over all 90 runs: mean goodput vs RTT inflation (left) and vs retransmitted bytes (right).

Per-destination summary over the 3 repetitions, destinations named by city (`tables/q3_dest_summary.md`, `tables/q3_per_run.md`; in-flight data is the median of unacked × MSS after the first 5 s):

Destination (min RTT)	CC	goodput Mbps (mean ± sd)	median srtt ms	RTT inflation	retrans %	in-flight MB	cwnd-limited %
Washington (26 ms)	cubic	234 ± 10	79	2.90	0.07	2.3	4
	reno	242 ± 16	81	3.01	0.06	2.3	2
	bbr	235 ± 7	74	2.79	0.02	2.3	2
Dallas (27 ms)	cubic	154 ± 15	59	2.09	0.09	1.0	14
	reno	150 ± 34	36	1.32	0.20	0.6	19
	bbr	240 ± 10	60	2.19	0.17	2.2	4
Ashburn (27 ms)	cubic	228 ± 13	71	2.57	0.11	2.2	6
	reno	235 ± 20	71	2.52	0.04	2.0	2
	bbr	240 ± 19	58	2.11	0.08	1.9	2
Seattle (53 ms)	cubic	225 ± 4	90	1.69	0.00	2.5	0
	reno	225 ± 5	91	1.70	0.00	2.5	0
	bbr	224 ± 5	83	1.58	0.00	2.5	2
Paris (97 ms)	cubic	185 ± 4	110	1.12	0.00	2.5	0
	reno	180 ± 4	115	1.11	0.00	2.6	0
	bbr	179 ± 2	112	1.13	0.00	2.6	2
Amsterdam (102 ms)	cubic	173 ± 2	117	1.14	0.00	2.5	0
	reno	175 ± 2	117	1.13	0.00	2.6	0
	bbr	174 ± 1	115	1.12	0.00	2.7	2
Naaldwijk (105 ms)	cubic	159 ± 26	118	1.09	0.00	2.6	3
	reno	175 ± 2	119	1.10	0.00	2.6	0
	bbr	170 ± 5	119	1.10	0.00	2.7	2
Zürich (117 ms)	cubic	148 ± 17	130	1.06	0.00	2.6	1
	reno	164 ± 2	128	1.07	0.00	2.7	0
	bbr	156 ± 4	130	1.06	0.00	2.7	2
Sydney (181 ms)	cubic	110 ± 2	189	1.03	0.02	2.7	1
	reno	79 ± 29	188	1.03	0.12	1.3	28
	bbr	102 ± 2	189	1.03	0.00	2.7	5
Singapore (208 ms)	cubic	83 ± 22	216	1.02	0.00	2.6	8
	reno	84 ± 9	250	1.18	0.04	2.7	1
	bbr	83 ± 5	220	1.05	0.00	2.7	2

Three operating regimes. The deciding quantity is how the host's in-flight cap (about 2.6 MB, from `tcp_wmem`; see Q2c) compares with the path's BDP. The bottleneck capacity C appears to be about 250–300 Mbps, the home access link: no run beat 307 Mbps at p95. That gives a crossover RTT of 2.6 MB × 8 / C ≈ 70–80 ms.

1. **Short paths, network-limited with a deep queue** (Washington, Ashburn and Seattle; min RTT 26–53 ms).
   - The BDP is only about 0.8 MB at 27 ms (240 Mbps × 27 ms), so the 2–2.5 MB in flight is far more than the pipe needs. The extra 1.5 MB sits in the bottleneck buffer.
   - The result is bufferbloat. srtt is 2.1–3.0 × min_rtt (58–80 ms against 27 ms), and srtt varies with a standard deviation of 18–24 ms. cwnd/BDP is 2–3.5 for every algorithm.
   - All three algorithms saturate the link at about 225–240 Mbps, and the throughput differences are smaller than the run-to-run spread.

- **BBR has the lowest RTT on 3 of these 3 paths** (inflation 2.11 vs 2.57/2.52 at Ashburn, 2.79 vs 2.90/3.01 at Washington, 1.58 vs 1.69/1.70 at Seattle), and the lowest retransmit rate at Washington.
- BBR's queue is still large. Linux BBR v1 caps in-flight data at 2 × its BDP estimate plus extra room for ACK aggregation, which Wi-Fi exaggerates. On top of that, the send-buffer cap is the binding limit here for every algorithm. Loss-based CUBIC and Reno simply fill whatever buffer there is.

2. **Short path with loss before the buffer fills** (Dallas, 27 ms).
   - This is the one destination where the algorithm changes throughput dramatically. **BBR gets 240 Mbps against 154 (CUBIC) and 150 (Reno).**
   - The path drops packets at 0.1–0.2%, with 2–4 Recovery episodes per run. Each loss makes CUBIC cut cwnd by 0.7× and Reno by 0.5×. Their in-flight data stays at 0.6–1.0 MB, about the 0.8 MB BDP, so they are cwnd-limited in 14–19% of samples (near 0% on the sender-limited long paths) and the link sits partly idle after each cut.
   - Q1's Reno panel shows the textbook sawtooth: linear ramps of +1 MSS per RTT ended by halvings.
   - BBR v1 does not reduce its rate on loss. It keeps 2.2 MB in flight and pays for it with a *higher* retransmit rate than CUBIC (0.17% vs 0.09%).
   - Reno has the lowest RTT here (inflation 1.32) only because it spends most of its time below the BDP.
3. **Long paths, sender-limited** (Paris, Amsterdam, Naaldwijk, Zürich, Sydney, Singapore; min RTT 97–208 ms).
   - 2.6 MB / RTT is below *C*, so the link is never full. There is almost no queue (inflation 1.02–1.14) and almost no loss (0.00–0.04% on all but one).
   - Goodput falls as 1/RTT: about 180 Mbps at 100 ms, 110 at 180 ms and 83 at 208 ms. That is the signature of a fixed window, not of the congestion controller. In-flight data is 2.5–2.7 MB for **every** algorithm, while cwnd is about 4000 (CUBIC, Reno) to 5600 (BBR) segments.
   - The three algorithms tie within noise (Amsterdam: 173 / 175 / 174 Mbps), and the controller only matters in two places:
     - *Startup.* CUBIC's HyStart left slow start early, with no loss, in 5 of the 30 CUBIC runs (ssthresh 29–339 segments). On a 100–200 ms path CUBIC then needs 10–20 s to grow to the cap. This is why CUBIC has the largest spread on these paths (sd 17–26 Mbps at Naaldwijk, Zürich and Singapore). Reno and BBR have no HyStart.
     - *Rare loss.* At Sydney, one loss burst at about 15 s halved Reno's cwnd repeatedly (1981 → 990 → 495 → 247). At 181 ms RTT, Reno's +1 MSS per RTT would need minutes to recover, so two reps averaged 58 and 67 Mbps and Reno's mean drops to 79 ± 29 against CUBIC's 110 ± 2. CUBIC copes better on long paths because its window growth is a function of time since the loss, not of the number of RTTs. With W_max ≈ 2000 segments it returns to W_max after $K = \sqrt[3]{W_{max}(1-\beta)/C} \approx 11$ s, whatever the RTT. Reno would need about 1000 RTTs (3 min at 181 ms) to regrow even a single halving.

Throughput vs RTT vs loss.

- *Throughput.* Across the 10 destinations no algorithm wins overall. Reno has the highest mean at 5, CUBIC at 3 and BBR at 2 (`tables/q3_winners.md`), but apart from Dallas (BBR +55%) and Sydney (Reno −28%) the gaps are within one standard deviation.
- *RTT.* BBR keeps the lowest RTT where a queue can form (the short paths).
- *Loss.* BBR accepts more retransmissions where the path is lossy.
- The trade-off plot shows the two failure modes as separate clusters:
  - top right: runs that saturate the link and pay 2–3× RTT inflation (all algorithms, short paths);
  - bottom left: runs with no inflation but a goodput set by the sender's window (long paths).

Variability. Each algorithm's spread across reps has a different cause:

- **BBR** is the most repeatable on long paths (sd 1–5 Mbps). Its main noise source is ProbeRTT, which drops cwnd to 4 segments for about 200 ms roughly every 10 s and is visible as the goodput dip at 11 s in the Amsterdam trace.
- **CUBIC's** spread comes from HyStart's startup decision.
- **Reno's** spread comes from whether a loss burst happens, and when.
- On the short paths all three vary by 5–20 Mbps between reps, because Wi-Fi capacity changes over the ~95 s between a destination's runs.

Follow-up: what changes when the send buffer stops being the limit. The main run showed that most destinations were limited by the host's send buffer. To see how the algorithms behave when cwnd is the limit, I measured 4 destinations twice, back to back, with identical settings except for `tcp_wmem`: max 32 MiB (`results/wmem32m/20261021-135240/`, 09:52–10:13 ET), then the default 4 MiB (`results/wmem4m/20261021-141430/`, 10:14–10:34 ET). That is 3 reps × 3 algorithms × 4 destinations each, 72 more tests, all successful. Running the 4 MiB baseline at the same time of day matters: compared with

the 4–5 AM main run, morning goodput at 4 MiB was up to 13% lower for most destination × algorithm pairs, and noisier.

Destination (min RTT)	CC	goodput 4 → 32 MiB (Mbps)	sd 4 → 32	RTT inflation 4 → 32	retrans % 4 → 32	in-flight MB 4 → 32
Seattle (54 ms)	cubic	196 → 166 (-15%)	28 → 38	1.74 → 1.93	0.08 → 0.43	2.4 → 2.4
	reno	222 → 129 (-42%)	51 → 50	1.74 → 2.09	0.01 → 1.06	2.5 → 1.5
	bbr	217 → 159 (-27%)	55 → 48	1.79 → 1.89	0.10 → 6.97	2.6 → 2.6
Amsterdam (103 ms)	cubic	163 → 193 (+18%)	7 → 15	1.15 → 1.39	0.00 → 0.40	2.6 → 4.0
	reno	157 → 147 (-6%)	12 → 23	1.24 → 1.19	0.02 → 1.73	2.7 → 2.4
	bbr	152 → 203 (+34%)	6 → 35	1.21 → 1.45	0.00 → 8.46	2.7 → 7.1
Zürich (118 ms)	cubic	150 → 177 (+18%)	11 → 20	1.12 → 1.32	0.00 → 0.35	2.7 → 3.7
	reno	151 → 150 (-1%)	11 → 72	1.11 → 1.10	0.00 → 1.58	2.7 → 2.6
	bbr	146 → 161 (+10%)	7 → 56	1.12 → 1.56	0.00 → 5.87	2.7 → 5.8
Sydney (181 ms)	cubic	95 → 190 (+99%)	18 → 55	1.05 → 1.12	0.20 → 0.96	2.7 → 5.0
	reno	108 → 153 (+41%)	2 → 45	1.04 → 1.07	0.00 → 2.67	2.7 → 3.6
	bbr	100 → 191 (+91%)	3 → 25	1.04 → 1.27	0.09 → 10.51	2.7 → 8.6

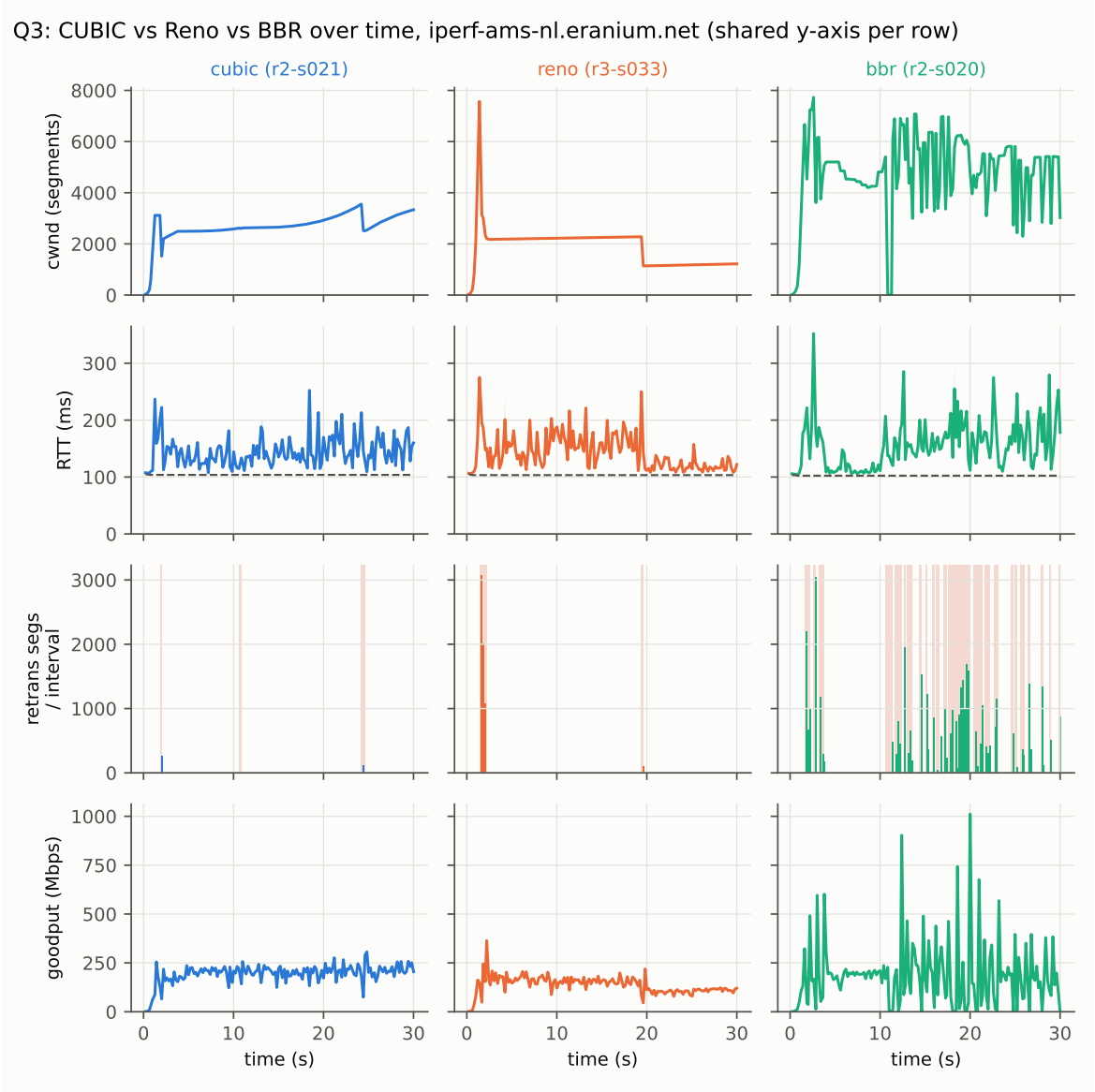


Figure 11: Q3 time series with `tcp_wmem` max = 32 MiB, Amsterdam, median run of each algorithm (shared y-axis per row).

- **On the long paths, the regime flips from sender-limited to network-limited and the algorithms separate.** In-flight data is no longer stuck at 2.6 MB:
  - BBR keeps 5.8–8.6 MB in flight;
  - CUBIC keeps 3.7–5.0 MB;
  - Reno keeps 2.4–3.6 MB.

  Goodput follows:
  - Sydney nearly doubles for CUBIC (95 → 190 Mbps) and BBR (100 → 191).
  - Amsterdam and Zürich gain 18% with CUBIC and 10–34% with BBR.
  - Reno gains only at Sydney (+41%) and is flat or slightly worse elsewhere.
- **CUBIC is the most effective loss-based algorithm on long paths.** Its total gain on the three long paths is almost as large as BBR's (+152 vs +157 Mbps), with a tenth of the retransmissions or less (0.35–0.96%), and its RTT inflation rises only moderately (1.12–1.39). The trace shows the reason: a ×0.7 cut followed by time-based regrowth, independent of RTT. HyStart, which hurt CUBIC in the main run, helps here. It ends slow start at a median cwnd of about 3700 segments, with a median burst of 713 retransmitted segments.
- **Reno is hurt by its own slow start.** Without HyStart it keeps doubling until the buffer overflows badly: a median peak cwnd of 8645 segments, about 5400 segments retransmitted in the first 5 s, and in the median run every one of its retransmissions in that window. It then halves its window and regrows by 1 MSS per RTT, which at 100–180 ms is too

slow to win back the lost ground in 30 s. That is the Reno trace above: a slow-start spike, a long flat line, and a second halving at 19.5 s.

- **BBR v1 buys throughput with loss and delay.**
  - It has the highest goodput on 2 of the 3 long paths, but retransmits **5.9–10.5%** of bytes.
  - It spends 22–47% of samples in Recovery, and its RTT inflation is the highest (1.27–1.56).
  - With 2 × BDP in flight on a path whose buffer cannot hold an extra BDP, every round overflows the queue. BBR v1 does not react to loss, so this never stops.
  - Its goodput trace swings between 0 and about 1000 Mbps. The upward spikes are an artifact of measuring goodput from `bytes_acked`: data that was SACKed during recovery is counted only when the cumulative ACK finally moves, all in one 200 ms interval. The mean over the test is still correct.
- **On the short path (Seattle, 53 ms), a bigger buffer only adds queue.** That path already filled the link with 4 MiB (inflation 1.74–1.79). With 32 MiB, every algorithm got *slower* (196–222 → 129–166 Mbps), with more delay (inflation 1.89–2.09) and loss (0.4–7%). This is bufferbloat at the sender: the extra window does not raise the bottleneck rate. It only overflows the router buffer.
- **Variability goes up for everyone.** Run-to-run sd on the long paths rises from 2–18 Mbps to 15–72 Mbps, because the timing of loss events now decides the result, whereas before a fixed window did.

Summary across all measurements.

Regime	Who wins	Why
Sender-limited: long RTT, default 4 MiB buffer	tie	the in-flight cap, not the CC, sets goodput = 2.6 MB / RTT
Network-limited with a deep queue (short RTT)	≈ tie on goodput; BBR lowest RTT	every algorithm fills the link; loss-based ones fill the buffer
Lossy path, loss before the queue fills (Dallas)	BBR (+55%)	CUBIC/Reno back off on non-congestive loss
Network-limited, long RTT (32 MiB buffer)	BBR CUBIC Reno on goodput; CUBIC best goodput per loss	Reno's ×0.5 and +1 MSS/RTT recovery is too slow; BBR ignores loss and retransmits 6–10%

Limitations.

- *One vantage point.* The bottleneck on the short paths is my home Wi-Fi/access link, so "who fills the queue" describes that one buffer.
- *Single flow.* Each test is one flow on a mostly idle path, so this compares how each algorithm uses the path, not how they compete with each other.
- *Small follow-up.* The send-buffer follow-up covers only 4 destinations and ran in the morning, not at night like the main run.
- *Few reps.* Three reps of 30 s are enough to see regimes but not to separate differences below about 10%.
- *Short tests.* 30 s is short for the long paths, where startup and recovery events are a large fraction of the run.
- *Server limits are unknown.* The public servers are shared, and their own receive limits and rates are unknown.
- *BBR version.* "BBR" here means Linux BBR v1, as shipped in kernel 6.8.

(b) A hand-written congestion control algorithm

Name: *QAIMD*, queue-aware AIMD with a BDP anchor.

What the measurements told me to fix.

1. *Loss-based algorithms fill whatever buffer there is.* On the short paths, CUBIC and Reno ran at 2.5–3.0 × min_rtt (30–50 ms of extra queue) and got no more throughput than a full pipe needs. → Aim for a small, explicit queue target, measured by delay.
2. *Treating every loss as congestion leaves the link idle.* At Dallas, 0.1–0.2% loss with little queue kept CUBIC and Reno at 150–154 Mbps while BBR got 240. But BBR ignores loss completely: it retransmitted the most there, and 6–10% of bytes once the send buffer was raised. → Back off on loss only when the loss comes with a queue or is heavy.

3. *Per-RTT additive increase is too slow on long paths.* After one loss burst at Sydney (181 ms), Reno was still far below its earlier rate when the test ended (79 ± 29 vs 110 Mbps). → After a reduction, return quickly to a BDP estimate instead of adding 1 MSS per RTT.
4. *RTT jitter breaks delay-based decisions.* srtt moved with a standard deviation of 3–9 ms on the uncongested long paths, and HyStart's delay check ended slow start early in 5 of 30 CUBIC runs, costing 10–20 s each. `tcpi_rttvar` (about 0.5 ms) does not see this jitter at all. Wi-Fi jitter only *adds* delay, so the smallest RTT in each round trip still shows the standing queue while ignoring the spikes. → Measure the queue as min-RTT-this-round − min_rtt, and leave startup only on a sustained signal.
5. *Slow start that only stops on loss overshoots badly.* With the 32 MiB buffer, Reno doubled to a median of 8645 segments and lost about 5400 in one burst. CUBIC's delay-based exit stopped near 3700. → Exit startup on a sustained queue or a bandwidth plateau, then drop straight to the BDP.
6. *BBR's ProbeRTT costs a full stall.* Dropping to 4 segments sent goodput to about 0 for 200 ms every 10 s. → Re-measure min_rtt by draining to 0.5 × BDP instead.

Signals. All of these are available to a Linux sender (they are the same `TCP_INFO` fields logged above):

- `min_rtt`: windowed minimum RTT over 10 s (propagation-delay estimate).
- `srtt`: smoothed RTT.
- `rtt_round_min`: the smallest RTT sample during the last round trip. Jitter only adds delay, so this keeps the standing queue and drops the spikes.
- `btl_bw`: windowed maximum delivery rate over 10 round trips, ignoring app-limited samples (bottleneck-rate estimate).
- `loss`: retransmitted bytes / delivered bytes in the last round trip (the loss proxy).

Derived from these:

- BDP = btl_bw × min_rtt.
- Queueing delay q = rtt_round_min − min_rtt.
- Queue threshold $q_{thr} = \text{clamp}(0.15 \cdot \text{min_rtt},\ 3\,\text{ms},\ 15\,\text{ms})$.

```
constants: BETA = 0.7, LOSS_TOL = 0.02, MIN_CWND = 4 MSS, PROBE_RTT_EVERY = 10 s

init: state = STARTUP; cwnd = 10 MSS; min_rtt = inf; btl_bw = 0

on every ACK:
    update srtt, rtt_round_min, min_rtt (10 s window)
    update btl_bw (max over 10 rounds, skip app-limited samples)
    # pace to spread bursts, but stay ahead of the ACK clock
    pacing_rate = 1.1 * cwnd / srtt

once per round trip (all data sent at the start of the round is ACKed):
    bdp   = btl_bw * min_rtt
    q     = rtt_round_min - min_rtt        # standing queue, Wi-Fi spikes removed
    q_thr = clamp(0.15 * min_rtt, 3 ms, 15 ms)
    loss  = retrans_bytes_this_round / delivered_bytes_this_round
    # loss + queue => congestion; loss without a queue => random loss, ignore
    congestive_loss = (loss > 0) and (q > q_thr or loss > LOSS_TOL)

    if state == STARTUP:
        if congestive_loss or (q > 2 * q_thr for 2 rounds)
                           or (btl_bw grew < 25% for 3 rounds):
            state = AVOID
            cwnd  = max(bdp, MIN_CWND)         # drop to pipe size: drains the queue
        else:
            cwnd = 2 * cwnd                    # exponential search for bandwidth

    else if state == AVOID:
        if congestive_loss:
            cwnd = max(BETA * cwnd, MIN_CWND)  # multiplicative decrease
        else if q > q_thr:
```

```
            # back off in proportion to the excess standing queue
            cwnd = max(cwnd * (1 - 0.5 * (q - q_thr) / srtt), MIN_CWND)
        else if cwnd < bdp:
            cwnd = cwnd + max(1 MSS, (bdp - cwnd) / 4)   # pipe not full: catch up
        else:
            cwnd = cwnd + 1 MSS                # additive increase: probe for more

    if min_rtt not refreshed for PROBE_RTT_EVERY:
        # drain the queue and re-measure min_rtt, without stalling
        hold cwnd = max(0.5 * bdp, MIN_CWND) for max(200 ms, 1 srtt)

on RTO:
    cwnd = MIN_CWND; state = STARTUP, capped at the last bdp   # don't overshoot

competitive mode (coexisting with loss-based flows):
    if q > q_thr for 20 rounds while our cwnd keeps shrinking and loss == 0:
        disable the delay back-off; act as AIMD(+1 MSS, x0.7) on congestive loss
        re-enable when q falls below q_thr
```

Design choices.

1. *Throughput.* `btl_bw` × `min_rtt` gives a BDP anchor. STARTUP exits to the BDP instead of the overshoot, and after any reduction the window climbs back to the BDP in a few RTTs (`(bdp-cwnd)/4`), not one MSS per RTT. This targets Reno's slow regrowth at Sydney and the idle link after each cut at Dallas.
2. *RTT.* The delay rule keeps the standing queue near q_{thr}, a few ms, instead of filling the buffer the way CUBIC/Reno do. That limits the RTT inflation we measured. The threshold scales with `min_rtt` so short paths are not forced into a large relative queue, and it is clamped to at least 3 ms. The queue is measured from the per-round minimum RTT, and startup exits only after two rounds of queue, so the Wi-Fi spikes that broke HyStart do not trigger false back-offs.
3. *Loss.* Only *congestive* loss, meaning loss together with a queue or above 2%, triggers the multiplicative decrease. Isolated retransmits with no queue growth are treated as random loss and ignored. That is the Dallas case, where Reno and CUBIC gave up about a third of the throughput. The retransmission rate stays bounded because a sustained loss rate above 2% still causes back-off, unlike BBR v1, which ignores loss entirely.
4. *Pacing* at 1.1 × cwnd/srtt avoids the line-rate bursts that overflow shallow buffers.

Fairness and convergence (competing QAIMD flows). When the queue is below threshold, every flow increases its window by the same additive step (1 MSS per round trip), and every reduction is *multiplicative*: ×0.7 on congestive loss, and ×(1 − 0.5(q − q_thr)/srtt) on queueing. Flows behind the same bottleneck see the *same* queueing delay at the same time, so they reduce together. By the Chiu–Jain argument, additive increase moves the allocation parallel to the fairness line, and multiplicative decrease moves it toward the origin. Each cycle shrinks the gap between two flows' windows by the decrease factor, so equal-RTT flows converge to an equal share.

The stable operating point is a small limit cycle in which the total in-flight data is about BDP $+ C \cdot q_{thr}$ (the link stays full, with only a few ms of queue). The equilibrium is set by the delay threshold, not by the buffer size, so unlike CUBIC/Reno it does not depend on how deep the router buffer is. The PROBE_RTT drain also helps: when one flow drains, every flow sees the lower RTT and refreshes its `min_rtt`. That stops a latecomer that measured an inflated `min_rtt` from claiming a larger share.

Assumptions.

- One bottleneck with a FIFO queue.
- `min_rtt` can be observed at least every 10 s.
- Delivery-rate samples are not app-limited, or are flagged as such (Linux exposes this flag).
- RTT noise is smaller than q_{thr}.
- Loss with no rise in RTT is not caused by congestion.

Limitations.

1. *RTT unfairness.* The additive step is per round trip, so with different RTTs the shorter-RTT flow still grows faster (as with Reno). One fix is to scale the step with `srtt`.

2. *Noisy delay signals.* Wi-Fi jitter, ACK aggregation and queueing on the reverse path can inflate `srtt` or `btl_bw`, causing false back-offs or overestimates.
3. *Coexistence with loss-based flows.* A delay-sensitive flow yields to CUBIC. Competitive mode only restores Reno-like aggressiveness, so CUBIC can still win on long-RTT paths.
4. *Shallow buffers.* Loss may come before the queue reaches q_{thr}, so the algorithm degenerates to AIMD with a BDP anchor.
5. *Untested on competing flows.* The constants are informed by single-flow measurements over one access link. The fairness and convergence claims are analytical: I did not run competing flows or a simulation.

Reproducibility

- Inputs: `servers.csv` (server list snapshot) and the command-line options in `README.md`. The Docker image installs Python 3.12, numpy, pandas and matplotlib from Ubuntu 24.04 apt packages.
- Host requirements: Linux kernel with `tcp_bbr` loaded (`sudo modprobe tcp_bbr`), Docker, and the container run with `--network host --cap-add NET_ADMIN` (in `run.sh`). Measured on host kernel `6.8.0-147-generic`.
- Commands: `./run.sh -n 10 -t 30 -i 0.2 -r 3 --seed 422` runs everything. The two follow-up runs use `results/<dir>/servers_control.csv` with `-n 4 -r 3 --seed 422`, after setting `net.ipv4.tcp_wmem` to `4096 16384 33554432` (32 MiB) or leaving the default `4096 16384 4194304` (see `README.md`). `./run.sh --analyze-only results/20261021-082724` regenerates every plot and table from the saved raw data.
- Caveat: public servers and Internet paths change, so a rerun will reproduce the method and the qualitative findings, not the exact numbers.

Use of AI tools

We used Claude Code (Anthropic's CLI) as a pair programmer. It helped us cross-check the iperf3 state machine and JSON framing against the official iperf3 source, write the first drafts of the Python modules and plotting code, set up the Multipass VM and Docker, and draft the prose in this report. We checked the output by running the client against real iperf3 servers, comparing the server-reported byte counts with `tcpi_bytes_acked`, checking `struct tcp_info` offsets against `linux/tcp.h`, and reading every plot. We reviewed all the code and the analysis, and we are prepared to explain the code, measurements and design choices in the oral exam.